

Student CGPA Management System

1. Project Overview

Project Description :

This is a C based student CGPA management system. It reads student information from and course grades from **student_input.txt** and calculated CGPA using course credits (CSE 4.5, MAT 3.0, ENG 3.0) and saves the final results in **output.txt**. It makes CGPA calculation easier, faster and more accurate.

2. Project Implementation

A. Defining the Data Structure

This section defines how each student's information is stored in the system's memory

```
#include <stdio.h>

struct student
{
    int stdID;
    char name[50];
    float cgpa_in_cse103;
    float cgpa_in_mat101;
    float cgpa_in_eng7101;
    float cgpa;
};
```

Logic: We use a **struct** to group student ID, full name, individual subject grades, and the final calculated CGPA under one identifier. This modular approach makes the code clean and allows us to easily handle an array of students.

B. Calculation Engine

This function handles the core logic of the program.

```
float calculateCGPA(float cse, float mat, float eng)
{
    float total_credits = 4.5 + 3 + 3;
    float sum = (cse * 4.5) + (mat * 3) + (eng * 3);
    return sum / total_credits;
}
```

Logic: The **calculateCGPA** function uses a weighted average formula: $(\text{Grade} * \text{Credits}) / \text{Total Credits}$. By assigning 4.5 credits to CSE and 3 credits to other subjects, the function ensures the CGPA reflects the actual weight of the courses.

C. File initialization and Safety Checks

Preparing the system to interact with external data.

```
FILE *input_file;
FILE *output_file;

struct student s[100];
int n = 0;

input_file = fopen("./student_input.txt", "r");

if (input_file == NULL)
{
    printf("No file found\n");
    return 1;
}

fscanf(input_file, "%d", &n);
```

Logic: This Program initializes the file pointers for input and output. A safety check (`input_file ==`

NULL) ensures that the program doesn't crash if the input file is missing, providing a user-friendly error message.

D. Data Retrieval and Processing Loop

Reading data from the file and calculating results iteratively.

```
for (int i = 0; i < n; i++)
{
    fscanf(input_file, "%d", &s[i].stdID);
    fscanf(input_file, " %[^\\n]", s[i].name);
    fscanf(input_file, "%f", &s[i].cgpa_in_cse103);
    fscanf(input_file, "%f", &s[i].cgpa_in_mat101);
    fscanf(input_file, "%f", &s[i].cgpa_in_eng7101);

    s[i].cgpa = calculateCGPA(s[i].cgpa_in_cse103, s[i].cgpa_in_mat101, s[i].cgpa_in_eng7101);
}

fclose(input_file);

output_file = fopen("./output.txt", "w");
```

Logic: A **for** loop iterates through the number of students specified in the file. It uses **fscanf** to grab records for each student and immediately calls the **calculateCGPA** function to fill the **struct** member CGPA.

E. Report Formatting and Final Output

Generating the final, human-readable code.

```
fprintf(output_file, "-----\\n");
fprintf(output_file, "%-8s %-15s %-10s %-10s %-10s %-8s\\n", "ID", "Name", "CSE103", "MAT101", "ENG7101",
"CGPA");
fprintf(output_file, "-----\\n");

for (int i = 0; i < n; i++)
{
    fprintf(output_file, "%-8d %-15s %-10.2f %-10.2f %-10.2f %-8.2f\\n",
        s[i].stdID, s[i].name, s[i].cgpa_in_cse103,
        s[i].cgpa_in_mat101, s[i].cgpa_in_eng7101, s[i].cgpa);
}

fprintf(output_file, "-----\\n");

fclose(output_file);
```

Logic: Finally, the program creates **output.txt** and writes a table-like header. It then loops

through array, printing each student's ID, Name, and grades into a perfectly aligned table using ***fprintf*** formatting tags.

3. Full Source Code

The complete source code can be accessed at: <https://github.com/protik-datta/student-management-system>